

MINT-VLA: MILESTONE INTEGRATION WITH NATIVE-SPACE TARGETS FOR VISION-LANGUAGE-ACTION MODELS

Anonymous authors

Paper under double-blind review

ABSTRACT

World-model-augmented robot policies condition action generation on predicted future observations or latent visual states. Pixel prediction is costly and includes control-irrelevant details, whereas precomputed targets can become inconsistent with a VLA encoder during fine-tuning. We introduce **MINT-VLA** (Milestone Integration with Native-Space Targets for Vision-Language-Action models), a conditioning adapter that predicts a future task-stage representation in a VLA’s current visual feature space. MINT-VLA identifies recurring task stages in successful demonstrations. During training, the policy encoder jointly encodes the observation and a future milestone image, and a compact predictor supplies the estimated milestone through the VLA’s native conditioning input. We instantiate this design as one prefix token in $\pi_{0.5}$. At inference, it adds 0.25% parameters without retrieval, a second visual encoder, or an image decoder. In a RoboTwin pilot, MINT-VLA improved a weak no-hint policy by 14.08 percentage points, but exceeded offline latent conditioning by only 0.83 points. At a fixed offline-conditioning checkpoint, removing the token reduced success from 70.67% to 67.17%, whereas current-state, feature-permuted, and cross-task replacements reached 71.00%, 71.50%, and 71.67%. Thus the visual token affects control, but current evidence does not show that predicted future content causes the improvement. MINT-VLA separates future-state prediction from architecture-specific policy conditioning.

[EXP-TODO T0–T2,T6,E1] Replace pilot evidence with the corrected multi-seed matrix, second-VLA evaluation, and completed matched-scene content interventions.

1 INTRODUCTION

World models represent environment dynamics by predicting future observations or latent states from the current state and, in many formulations, an action. Recent vision-language-action (VLA) systems use predicted images, latent observation features, or visual subgoals to inform action generation (Hu et al., 2024; Black et al., 2024; Syed et al., 2026; Xu et al., 2026). Prediction quality alone, however, does not establish an improvement in control. WorldArena reports a correlation of only $r=0.360$ between world-model quality and planning performance (Shang et al., 2026), while VLA prediction systems use specialized attention, alignment, or gating before future supervision reaches the controller (Zhang et al., 2025; Xu et al., 2026; Zhu et al., 2026). The unresolved problem is therefore not only how to predict a future state, but how to integrate the prediction into a pretrained action policy.

This integration problem is especially important when the VLA visual encoder remains trainable. An independent visual world model preserves its own representation but repeats perception or generation at deployment. A precomputed latent is cheaper, yet an external encoder can organize the scene differently from the policy. Even a feature computed by the policy encoder becomes outdated as fine-tuning changes that encoder. Finally, an auxiliary future loss can alter shared current features whether or not the policy uses the predicted

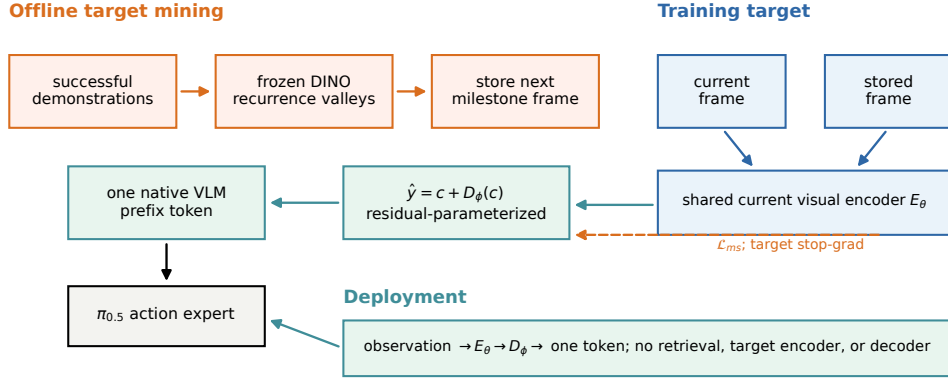


Figure 1: $\pi_{0.5}$ instantiation of MINT-VLA. Recurrence mining selects a later milestone frame from successful demonstrations. During training, the current policy encoder defines a stop-gradient target and a residual-parameterized predictor produces an absolute milestone feature. The $\pi_{0.5}$ adapter maps that feature to one prefix token.

content. Encoder mismatch, target drift, forward conditioning, and backward representation shaping are different causes; a method-versus-baseline comparison cannot identify them.

Here we present **MINT-VLA** (Milestone Integration with Native-Space Targets for Vision-Language-Action models), a compact adapter that defines milestone targets in the policy’s current visual space (Fig. 1). MINT-VLA first finds stage states that recur across successful demonstrations and stores the corresponding frames. During policy training, the current encoder represents both the observation and a stop-gradient milestone target. A two-layer predictor estimates their displacement and reconstructs an absolute milestone feature. An architecture-specific projection passes this feature through the VLA’s native conditioning input; our $\pi_{0.5}$ implementation appends one prefix token. At deployment, the predictor operates on the current observation; it requires no retrieval, target encoder, or image decoder.

The completed experiments separate the effect of adding the token from the effect of its predicted milestone content. In a legacy matched RoboTwin pilot, offline absolute conditioning improved a weak no-hint $\pi_{0.5}$ by 13.25 percentage points and MINT-VLA improved it by 14.08 points. Their 0.83-point difference came from one training seed and varied in sign across tasks. More importantly, a fixed A2 checkpoint achieved 70.67% with its predicted token and 67.17% with a zero token, but current, feature-permuted, and cross-task tokens reached 71.00%, 71.50%, and 71.67%. Visual conditioning therefore helped relative to removing the token, but the current evidence does not establish that the predicted future state improves action selection. A public $\pi_{0.5}$ reached 78.42% through the same evaluator, which further makes corrected matched training necessary.

[EXP-TODO T0–T2,E1] Replace this legacy evidence paragraph with corrected multi-seed task effects and Holm-adjusted matched-scene interventions.

This paper makes three contributions:

- We introduce a milestone target recomputed by the current policy encoder and an adapter that uses each VLA’s native conditioning input; the evaluated $\pi_{0.5}$ implementation adds one token and 0.25% parameters.
- We define controlled comparisons for target recomputation, predictor parameterization, policy conditioning, and auxiliary-gradient routing.
- We report task-level pilots and fixed-checkpoint interventions that show when an extra visual token is useful without attributing that gain prematurely to predicted future content.

2 RELATED WORK

Predictive representations for robot policies. Prior systems predict future images, videos, latent observation features, or visual subgoals. VPP conditions inverse dynamics on future visual representations produced by a video diffusion model (Hu et al., 2024); SuSIE generates image subgoals for a goal-conditioned controller and compares them with ground-truth future images (Black et al., 2024). WorldVLA predicts future images together with actions, whereas DreamVLA forecasts dynamic, spatial, and semantic scene information for action prediction (Zhang et al., 2025; Cen et al., 2025). AHEAD predicts future patches in a frozen OpenVLA feature space (Syed et al., 2026). FutureVLA learns joint visuomotor representations and transfers them to downstream VLAs through latent alignment (Xu et al., 2026). Concurrent Δ VLA models changes relative to the current scene (Zhu et al., 2026). LaWAM predicts action-conditioned future observation features as latent visual subgoals; WLA-0 jointly predicts subtasks, images, and actions; and Enfold distills a generative world model into a representation computed from the current observation (Chen et al., 2026; Yang et al., 2026; Zeng et al., 2026). MINT-VLA therefore does not claim the first latent future state, visual subgoal, residual predictor, or auxiliary prediction loss. It addresses a specific integration problem: keeping target features consistent with a trainable policy encoder and mapping the prediction to the VLA’s existing conditioning interface.

Milestone selection. States shared across successful trajectories have long been used as bottlenecks or options (Şimşek & Barto, 2008; Sutton et al., 1999). MINT-VLA uses cross-episode visual recurrence to select candidate stage states without manually annotated boundaries. This differs from a fixed temporal horizon and from terminal-goal conditioning, but recurrence itself is not sufficient evidence that the selected state is useful. Our planned fixed-horizon, random, terminal, and oracle controls follow SuSIE’s principle of separating subgoal quality from controller capacity.

Integration and negative transfer. Auxiliary objectives can improve or damage a shared representation depending on weighting and task (Du et al., 2018; Lin et al., 2019; Yu et al., 2020); negative transfer is not explained by gradient conflict alone (Jiang et al., 2023; Lyle et al., 2021). VLA systems accordingly use different interfaces, including structured attention, latent alignment, conditional variation modules, or a frozen policy feature space (Zhang et al., 2025; Xu et al., 2026; Zhu et al., 2026; Syed et al., 2026). Selective gradient routing is also used to preserve pretrained VLA semantics (Driess et al., 2025). MINT-VLA’s current implementation stops the milestone target branch but still allows its loss to update current visual features through the predictor; the planned auxiliary-only, prefix-only, and detached-current controls measure those paths rather than assuming isolation.

Establishing control benefit. Prediction quality and control performance can diverge (Shang et al., 2026; Jiang et al., 2026; Yu et al., 2026), and correlated conditioning can induce causal confusion (de Haan et al., 2019; Wen et al., 2020). Evaluation seeds also do not replace independent training seeds (Henderson et al., 2018; Agarwal et al., 2021). We therefore require closed-loop success, fixed-checkpoint content interventions, task-level reporting, and training-seed replication. These controls are necessary to determine whether performance depends on the predicted future state rather than merely on an additional visual token.

3 PROBLEM SETTING AND CONTROLLED COMPARISONS

Let $h_t = E_\theta(o_t)$ be the current policy representation, y_t the feature of a future milestone frame, and q_t the token presented to the action policy:

$$a_t = \pi_\psi(h_t, q_t, l), \quad (1)$$

where l is the task instruction. When E_θ changes during fine-tuning, future conditioning introduces four separable design choices.

Target selection. A fixed horizon, terminal frame, random future, or recurrent milestone encodes a different claim about which future state is useful.

Table 1: Evidence required for each claim. Pending rows are not inferred from the legacy pilot.

	Claim	Required comparison	Status
Q1	matched control performance	corrected-recipe A0 vs. A2-Abs vs. MINT-VLA	TODO T0-T2
Q2	correct content guides action	fixed checkpoint: correct/current/zero/instance-shuffle/cross-task	TODO E1
Q3	effect of target and integration design	one-factor saved/recomputed, absolute/residual, auxiliary/policy-input controls	TODO T3
Q4	robustness, task scope, and efficiency	train seeds, mature policy, predeclared tasks, cost	TODO T4-T5, E2
Q5	use with a second VLA architecture	same target protocol with the second VLA’s native input	TODO T6

Representation space. The target may come from an external encoder, an earlier policy encoder, or the current encoder. A projection matches dimensions but does not align semantic neighborhoods, and saved features do not follow encoder drift.

Parameterization and forward path. The model may predict an absolute feature y_t or a displacement $y_t - h_t$, and may inject that prediction into the policy or use it only as an auxiliary objective. A residual target can emphasize change without being a natural standalone conditioning input.

Backward path. With milestone loss $\mathcal{L} = \mathcal{L}_{\text{act}} + \lambda \mathcal{L}_{\text{ms}}$, the auxiliary gradient can reshape shared current features. A performance change may thus come from forward guidance, representation shaping, or both.

Table 1 specifies the minimum comparison for each paper claim. Simulator seeds estimate rollout variation for one checkpoint; only independent training seeds measure optimization robustness.

The legacy A0–A3 matrix is retained as exploratory evidence because its arms are internally matched, but it cannot close Q1 under the intended $\pi_{0.5}$ training setting. LaWAM and LIBERO are supporting diagnostics only: the former lacks a clean VLA-from-base comparison, while the latter is near saturation for the policies studied here.

4 MINT-VLA

4.1 OVERVIEW

MINT-VLA separates future-state supervision from the mechanism used by a particular VLA to receive additional conditioning. It has four components: (i) milestone images selected from successful demonstrations, (ii) target features computed by the VLA’s current visual encoder, (iii) a predictor that estimates a milestone feature from the current observation, and (iv) a small architecture-specific adapter that passes this feature to the policy. The demonstrations and milestone images are used only for training. Deployment retains the policy encoder, milestone predictor, and adapter.

4.2 MILESTONE TARGET DISCOVERY

Successful demonstrations repeatedly pass through recognizable task stages: an object is grasped, one block is placed, or a handover is completed. For a frame o_t^i from episode i , we L2-normalize a frozen DINOv3 pooled feature $f(o_t^i)$ and compute

$$r(o_t^i) = \frac{1}{|\mathcal{E}| - 1} \sum_{j \neq i} \max_s \exp\left(-\frac{\|f(o_t^i) - f(o_s^j)\|_2^2}{2\sigma^2}\right), \quad (2)$$

where σ is the median cross-episode nearest-neighbor distance. We Gaussian-smooth each episode’s score ($\sigma_t=1.4$), split it at valleys with prominence 0.03 and temporal separation $\max(2, \lfloor T/12 \rfloor)$, and select the highest-scoring frame in each segment. Each frame is paired with the representative of the next segment, or with the final frame after the last boundary. We process at most 200 uniformly sampled successful episodes per task. This procedure uses task grouping and success labels, but no manually annotated stage boundaries.

4.3 NATIVE-SPACE TARGET CONSTRUCTION

Let E_θ denote the visual encoder of the VLA being trained. MINT-VLA stores the milestone image o_t^{ms} , rather than a feature computed before policy training. At each update, the current encoder maps the observation and milestone image into the same representation space:

$$c_t = \text{pool}(E_\theta(o_t)), \quad y_t = \text{sg}[\text{pool}(E_\theta(o_t^{\text{ms}}))]. \quad (3)$$

We call y_t a *native-space target* because its coordinates are defined by the current policy encoder, not by an external encoder or an earlier encoder snapshot. Stop-gradient prevents the target branch from changing E_θ , while allowing the target itself to follow changes in the encoder over training.

4.4 MILESTONE PREDICTION

A two-layer predictor estimates a change from the current feature and reconstructs an absolute milestone representation:

$$\hat{y}_t = c_t + D_\phi(c_t), \quad \mathcal{L}_{\text{ms}} = \|\hat{y}_t - y_t\|_2^2. \quad (4)$$

Thus, “residual” describes the predictor parameterization; the policy receives the absolute prediction $\hat{y}_t$. The current predictor is conditioned on the observation, while the task instruction is supplied separately to the VLA. This design cannot distinguish instruction-dependent futures that share the same visual observation. Moreover, stop-gradient protects only the target branch: $\nabla_\theta \mathcal{L}_{\text{ms}}$ can still pass through c_t . We therefore do not describe the current method as gradient-isolated.

4.5 POLICY-SPECIFIC CONDITIONING ADAPTER

The predicted milestone is connected to a policy through an architecture-specific adapter A_ω :

$$q_t = A_\omega(\hat{y}_t), \quad a_t = \pi_\psi(o_t, l, q_t), \quad (5)$$

where l is the instruction and q_t follows the conditioning format exposed by the VLA. For a token-based policy, A_ω projects the milestone into a conditioning token. A policy with an existing cross-attention or latent conditioning input can instead map $\hat{y}_t$ to that interface. Milestone selection, native-space supervision, and the residual prediction objective are shared across implementations. The visual encoder and adapter come from the target VLA, and the predictor is sized to that encoder’s feature width. This separation makes MINT-VLA compatible by design with multiple VLA interfaces, but the present experiments establish performance only for the $\pi_{0.5}$ instantiation.

[EXP-TODO T6] Train and evaluate the same milestone-selection and native-space prediction protocol on at least one additional VLA using its own feature width and conditioning adapter.

4.6 $\pi_{0.5}$ INSTANTIATION

$\pi_{0.5}$ contains a PaliGemma visual-language prefix and an action expert. We implement A_ω as a linear projection to the prefix width and append one bidirectionally visible milestone token:

$$q_t = P_\omega(\hat{y}_t), \quad X_{\text{prefix}} = [X_{\text{image}}, q_t, X_{\text{text}}]. \quad (6)$$

The milestone-loss weight is 0.05. This implementation adds no cross-attention tower, second visual encoder, image decoder, or demonstration retrieval at deployment. It adds 0.25%

parameters to the evaluated $\pi_{0.5}$ policy. The A0–A3 comparison arms and their matched training configuration are defined in Section 5; complete implementation values are listed in Appendix A.

5 EXPERIMENTAL DESIGN

5.1 BENCHMARK, EVALUATOR, AND EVIDENCE TIERS

RoboTwin 2.0 is the primary benchmark. We evaluate hammering, ranking by color, ranking by size, handover, stacking two blocks, and stacking three blocks. Each completed pilot cell contains four simulator seed groups and 50 accepted expert-feasible episodes per group (200 episodes per task and 1,200 per method). Policies predict 50 actions and replan after executing 36. Accepted scene identifiers must be intersected across conditions for paired tests. LIBERO is reported only as an auxiliary saturation and negative-transfer diagnostic.

We separate three evidence tiers. The *external calibration* is the published $\pi_{0.5}$ checkpoint evaluated through our bridge. The *legacy pilot* starts every arm from `pi05_base` but uses joint-delta actions, quantile normalization, batch 64, and 20k updates. The *confirmatory matrix* will use absolute actions, mean/std normalization, batch 16, 50k updates, and the published optimizer schedule for every arm. Only the confirmatory matrix can support the final Q1 claim.

5.2 Q1: MATCHED CONTROL PERFORMANCE

The claim-bearing comparison is A0 (no hint), A2-Abs (offline absolute milestone), and A3/MINT-VLA (current-encoder residual predictor), trained from the same initialization and data under the confirmatory recipe. A0 has no hint field, milestone loss, or milestone-specific parameters. We will report each training seed, the seed mean and interval, and all task-level results.

[EXP-TODO T0–T2] Finish corrected A0; after its 70% operational gate, train matched A2-Abs and six-task-coverage A3 for three independent seeds.

5.3 Q2: EFFECT OF THE PREDICTED TOKEN CONTENT

For each trained A2-Abs and A3 checkpoint, we hold weights fixed and replace the correct prediction with (i) the current-state feature, (ii) zeros, (iii) a within-task shuffled future, and (iv) a cross-task future. A completed feature-dimension permutation is reported separately because it tests feature structure, not instance relevance. Conditions use the same accepted scene identifiers. Correct-versus-current tests temporal content; correct-versus-shuffled tests instance relevance while preserving a plausible feature distribution; zero tests removal of the token; cross-task tests semantic misdirection. We report paired success differences and exact McNemar tests with Holm correction across planned contrasts.

[EXP-TODO E1] Complete A3 zero, instance-shuffled, and cross-task conditions; run the missing A2 instance shuffle; freeze scene intersections and report discordant counts with Holm-adjusted exact McNemar tests.

5.4 Q3: FACTORIZED INTERFACE MECHANISM

The A2-to-A3 pilot changes how target features are computed, predictor capacity, auxiliary loss, and the injected value at once. The confirmatory ablation therefore holds predictor architecture, loss, injected value, and parameter count fixed while varying one factor:

Factor	Controlled comparison
target computation	saved target feature vs. frame re-encoded by the current encoder
predictor form	direct absolute predictor vs. residual-parameterized predictor, both injecting absolute
forward guidance	auxiliary-only/no-prefix vs. predicted prefix
backward shaping	prefix-only/no-aux and detached-current auxiliary branch
milestone selection	fixed horizon, random future, terminal frame, and oracle milestone

[EXP-TODO T3] Train the one-factor target-computation, predictor-form, forward/backward route, capacity, and instruction-aware controls.

5.5 Q4: ROBUSTNESS, SCOPE, AND EFFICIENCY

Training seed is the top-level uncertainty unit. We will use a hierarchical bootstrap over training seed, task, and paired scene, while retaining task-level values to expose heterogeneous effects. Transfer is tested by matched fine-tuning from a mature initialization, not by comparing against an unrelated public checkpoint. The task-scope hypothesis is predeclared on block and bowl stacking, with reactive/contact and fine-grained geometric tasks as controls. We will also report added parameters, training FLOPs, peak memory, inference latency, and throughput relative to A0.

Training-seed, mature-initialization, and predeclared task-group experiments remain confirmatory gates. We report static deployment cost now and reserve measured memory and latency claims for a paired benchmark.

[EXP-TODO T4–T5,E2] Train the predeclared task panel and mature-initialization matrix; measure paired peak memory, inference latency, and server throughput.

5.6 Q5: INSTANTIATION ON A SECOND VLA ARCHITECTURE

The milestone target and predictor are defined independently of the $\pi_{0.5}$ prefix. To test use with another architecture, we will retain the milestone selection, native-space objective, data, and evaluation protocol; size the predictor to the second VLA’s visual feature width; and connect it through that VLA’s native conditioning input. The identifying comparison is MINT-VLA versus a matched no-milestone baseline within the second architecture, not raw success differences between two VLA families.

[EXP-TODO T6] Report matched no-milestone and MINT-VLA results for at least one additional VLA; do not claim multi-architecture evidence from the $\pi_{0.5}$ experiment alone.

6 RESULTS

6.1 CALIBRATION: THE EVALUATOR SUPPORTS A MATURE $\pi_{0.5}$

The published $\pi_{0.5}$ checkpoint obtains 78.42% on the same six-task, four-seed bridge (941/1,200 successes): 93.0 hammer, 96.5 ranking-RGB, 64.0 ranking-size, 53.5 handover, 93.5 Stack-2, and 70.0 Stack-3. This result is an evaluator calibration, not a matched baseline, because its training mixture and recipe differ. The calibration nevertheless shows that the pilot A0 score is mainly a policy-training issue rather than a globally broken evaluation bridge.

6.2 LEGACY MATCHED PILOT: USEFUL SIGNAL, INSUFFICIENT BASELINE

Table 2 reports the completed legacy pilot. Every conditioned method improves its six-task macro over A0. A2-Abs gains 13.25 points and A3 gains +14.08 points. However, the absolute success levels remain far below the public calibration, each row has one training

seed, and A3 exceeds A2-Abs by only +0.83 points. We therefore use this result to choose the confirmatory arms, not as the final MINT-VLA performance claim.

Table 2: Legacy matched $\pi_{0.5}$ RoboTwin pilot: joint-delta actions, quantile normalization, batch 64, 20k updates. Each cell averages 200 accepted episodes; each row has one training seed.

Method	Hammer	Rank RGB	Rank size	Handover	Stack-3	Stack-2	Macro
A0: no hint	70.0	50.5	24.0	9.0	17.5	42.0	35.50
A1: external absolute	79.5	52.5	24.5	5.5	22.0	63.0	41.17
A2: offline absolute	78.0	65.0	30.5	19.5	30.5	69.0	48.75
A2: offline residual	76.5	52.5	34.5	18.0	25.0	56.5	43.83
A3: MINT-VLA current-encoder	66.0	70.5	25.5	26.0	35.5	74.0	49.58

The task profile is heterogeneous (Fig. 2). A3 improves ranking-RGB, handover, and stack-ing but loses on hammer and ranking-size versus A2-Abs. Offline residualization is 4.92 points below offline absolute, so “more change-focused” is not equivalent to “more useful as a policy token.” The A2-to-A3 contrast further changes target computation, predictor capacity, auxiliary loss, and the injected value; it motivates Q3 but does not identify which factor caused the difference.

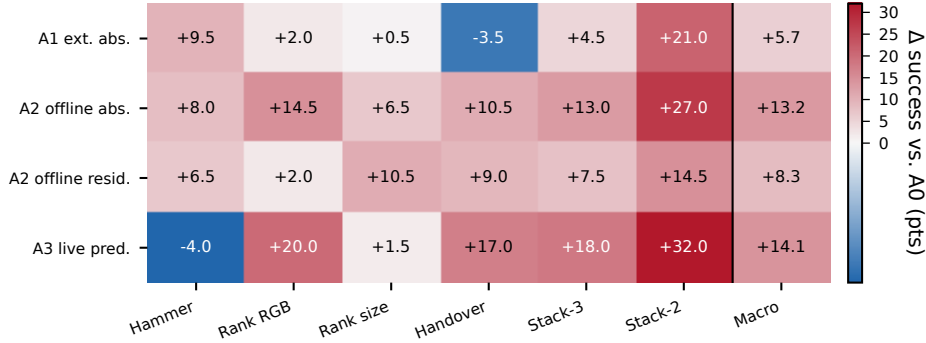


Figure 2: Legacy-pilot task deltas relative to A0. Each cell contains 200 accepted episodes from one trained checkpoint. The methods shift gains across tasks; none dominates every task.

6.3 THE CORRECTED MATCHED COMPARISON IS PENDING

The corrected A0/A2-Abs/A3 matrix is the primary missing result. Table 3 is intentionally reserved so that pilot numbers cannot silently become the final baseline.

Table 3: Confirmatory public-recipe RoboTwin matrix. Report mean and interval over independent training seeds, plus the full task breakdown.

Method	Train seeds	Six-task macro	Δ vs. A0
A0 no hint	TODO T0,T2	TODO T0,T2	—
A2 offline absolute	TODO T1-T2	TODO T1-T2	TODO T1-T2
A3 MINT-VLA current-encoder	TODO T1-T2	TODO T1-T2	TODO T1-T2

6.4 THE VISUAL TOKEN HELPS, BUT ITS PREDICTED FUTURE CONTENT DOES NOT YET

The five completed A2 conditions do not show that predicted future content improves control on the three selected tasks. Correct, current-state, zero, feature-permuted, and cross-task content obtain 70.67, 71.00, 67.17, 71.50, and 71.67% over 600 episodes. A nonzero visual token helps relative to removing the token, but the correct prediction is not better than any

structured nonzero replacement in aggregate. The task profile differs: correct/current/zero are 78.0/73.0/69.5% on hammer, 65.0/68.5/62.0% on ranking-RGB, and 69.0/71.5/70.0% on Stack-2. Across 179 matched hammer scenes, correct beats zero in 38 discordant cases and loses in 22 (exact two-sided McNemar $p=0.0519$, before multiplicity correction). This evidence supports visual token input but not a general benefit from predicted milestone content.

Table 4: Fixed-checkpoint content intervention. Final cells use identical accepted scenes and Holm-adjusted paired tests.

Checkpoint	Correct	Current	Zero	Permuted	Cross-task
A2-Abs	70.67 [†]	71.00 [†]	67.17 [†]	71.50 [†]	71.67 [†]
A3	70.17 [†]	70.50 [†]	TODO E1	TODO E1	TODO E1

[†]Three-task aggregate over 600 episodes; not directly comparable to the six-task pilot macro.

[EXP-TODO E1] Add within-task instance shuffling for A2; finish A3 zero, permuted, and cross-task conditions; replace aggregate comparisons with matched-scene discordant counts and Holm-adjusted tests.

6.5 THE PILOT DOES NOT ISOLATE THE EFFECT OF TARGET RECOMPUTATION

No completed comparison holds capacity, target, loss, and injected value fixed while changing only how the target feature is computed. Likewise, the current pilot does not separate forward conditioning from auxiliary representation shaping. The current evidence therefore supports A3 as an unfactorized combined integration design, but does not isolate current-encoder target recomputation or residual parameterization as its cause.

The remaining training matrix tests saved versus recomputed target features, direct-absolute versus residual-parameterized prediction, auxiliary-only, prefix-only, detached-current, fixed-horizon, random-future, terminal-frame, and oracle-milestone controls.

[EXP-TODO T3] Populate the factorized mechanism matrix with newly trained, parameter-matched controls and replicate every claim-bearing contrast.

6.6 TASK SCOPE AND OPTIMIZATION ROBUSTNESS REMAIN UNRESOLVED

The legacy simulator seeds quantify rollout variation around one trained checkpoint, not optimization variance. The ordered-construction hypothesis is also post hoc: on Stack-2/3, legacy A3 averages 54.75% versus 29.75% for A0, but the public unconditioned checkpoint already reaches 81.75%. This may reflect useful stage structure, baseline room for improvement, or both. We will make a task-specific claim only if the direction replicates on predeclared block and bowl stacking, beats a fixed-horizon target encoded by the current policy, and differs from reactive/contact and geometric control tasks.

Independent training seeds, matched mature-initialization transfer, and the predeclared task panel remain pending. Exact parameter counts and static predictor FLOPs are reported in the appendix; paired memory and latency measurements will accompany the confirmatory checkpoints.

[EXP-TODO T4–T5,E2] Populate the predeclared scope, mature-transfer, and paired deployment-cost results.

6.7 USE WITH A SECOND VLA ARCHITECTURE REMAINS TO BE TESTED

The current RoboTwin experiment instantiates MINT-VLA only on $\pi_{0.5}$. It tests the one-token prefix implementation, not its use with another VLA architecture. A matched experiment on a second VLA is required before the paper claims evidence beyond the $\pi_{0.5}$ implementation.

[EXP-TODO T6] Add a matched second-VLA baseline and MINT-VLA result using the same milestone construction and evaluation protocol.

6.8 SUPPORTING BOUNDARY DIAGNOSTICS

LaWAM interventions show why Q2 is necessary: correct hints are not reliably better than zero, instance-shuffled, or cross-task hints, and the former “no-WM” arm inherits WM-aware pretraining and current DINO conditioning. It is therefore named *LaWAM-init/Future-off* and used only to diagnose downstream integration. LIBERO is similarly auxiliary: its near-saturated four-suite aggregate is neutral to negative under milestone conditioning. Full tables and multiplicity-aware analyses belong in Appendix A; neither diagnostic substitutes for the missing $\pi_{0.5}$ gates.

7 DISCUSSION AND CONCLUSION

The current results distinguish a useful visual interface from useful future content. Adding a nonzero visual token improved the fixed A2 checkpoint over a zero token by 3.50 percentage points on the three-task panel. Replacing the predicted future with the current feature changed success by only 0.33 points in the opposite direction. The intervention therefore supports use of the visual token, but not a claim that the predicted milestone improves control across tasks or task categories. Hammering showed the expected ordering, whereas ranking-RGB and Stack-2 did not. This heterogeneity is a result to explain, not variation to hide in a macro average.

MINT-VLA addresses two concrete representation problems. Re-encoding stored frames prevents target features from becoming outdated as the policy encoder changes, while a small policy-specific adapter avoids deploying a second visual stack. The evaluated $\pi_{0.5}$ adapter uses one prefix token. The design does not by itself solve auxiliary-objective interference: the target branch is stopped, but the milestone loss can still update the encoder through the current feature. Nor does residual parameterization guarantee a useful policy input. The legacy comparison changed target computation, capacity, loss, and parameterization together, so only the factorized controls can assign credit to an individual mechanism.

The pilot has three further limitations. Its no-hint policy used a weak joint-delta, quantile-normalized 20k-step recipe, and every arm used one training seed. The milestone-pair artifact also omitted Stack-3; on that task the predictor was trained through the action loss without milestone auxiliary supervision. Finally, the observation-only predictor cannot distinguish two instructions that require different futures from the same scene. These limitations rule out claims of broad superiority, recurrence-based gains on Stack-3, or instruction-aware milestone selection.

The central hypothesis remains precise: a future condition is useful when its content adds decision-relevant information beyond the current visual feature, and when the interface preserves that information in the policy’s changing representation space. MINT-VLA supplies one implementation of this hypothesis; the corrected multi-seed matrix, content interventions, and one-factor ablations determine whether its predicted milestone satisfies it. This framing turns future conditioning from an architectural label into a claim that can be tested against current-state, instance-shuffled, capacity-matched, and gradient-route controls.

[EXP-TODO T0–T6,E1–E2] Rewrite the discussion around the corrected multi-seed effect, completed content interventions, isolated mechanism, predefined task panel, and measured deployment cost.

REPRODUCIBILITY STATEMENT

The anonymized supplementary package includes preprocessing and evaluation code, effective training configurations, checkpoint identifiers, and aggregate evaluation reports. Appendix A records source and data hashes, software versions, simulator seeds, and the currently available artifact coverage. Accepted-scene manifests and per-episode outcomes will accompany the confirmatory matrix so that all paired tests can be reproduced.

[EXP-TODO T0–T6,E1] Replace provisional reproducibility text with the final anonymous archive URL and complete experiment manifests.

AI USAGE DISCLOSURE

Generative AI tools assisted literature discovery, hypothesis refinement, experimental design, implementation and code review, figure and table formatting, drafting and translation, and interpretation of experimental results. They were not used to generate demonstrations, simulator outcomes, or reported measurements. The authors inspected the relevant source code, checked configuration and artifact provenance, ran builds and tests, and verified paper claims against the underlying reports. The authors take responsibility for all content and artifacts in this submission.

REFERENCES

- Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, 2021.
- Kevin Black, Mitsuhiko Nakamoto, Pranav Atreya, Homer Walke, Chelsea Finn, Aviral Kumar, and Sergey Levine. Zero-shot robotic manipulation with pre-trained image-editing diffusion models. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/8ea50bf458f6070548b11babbe0bf89b-Abstract-Conference.html.
- Jun Cen, Chaohui Yu, Hangjie Yuan, Yuming Jiang, Siteng Huang, Jiayan Guo, Xin Li, Yibing Song, Hao Luo, Fan Wang, Deli Zhao, and Hao Chen. Worldvla: Towards autoregressive action world model. *arXiv preprint arXiv:2506.21539*, 2025.
- Jialei Chen, Kai Wang, Kang Chen, Shuaihang Chen, Feng Gao, Wenhao Tang, Zhiyuan Li, Weilin Liu, Zhuyu Yao, Boxun Li, Yuanbo Xu, and Chao Yu. Lawam: Latent world action models for efficient dynamics-aware robot policies. *arXiv preprint arXiv:2606.15768*, 2026.
- Pim de Haan, Dinesh Jayaraman, and Sergey Levine. Causal confusion in imitation learning. In *Advances in Neural Information Processing Systems*, 2019.
- Danny Driess, Jost Tobias Springenberg, Brian Ichter, Lili Yu, Adrian Li-Bell, Karl Pertsch, Allen Z. Ren, Homer Walke, Quan Vuong, Lucy Xiaoyang Shi, and Sergey Levine. Knowledge insulating vision-language-action models: Train fast, run fast, generalize better. *arXiv preprint arXiv:2505.23705*, 2025.
- Yunshu Du, Wojciech M. Czarnecki, Siddhant M. Jayakumar, Mehrdad Farajtabar, Razvan Pascanu, and Balaji Lakshminarayanan. Adapting auxiliary losses using gradient similarity. *arXiv preprint arXiv:1812.02224*, 2018.
- Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- Yucheng Hu, Yanjiang Guo, Pengchao Wang, Xiaoyu Chen, Yen-Jen Wang, Jianke Zhang, Koushil Sreenath, Chaochao Lu, and Jianyu Chen. Video prediction policy: A generalist robot policy with predictive visual representations. *arXiv preprint arXiv:2412.14803*, 2024.
- Feng Jiang, Yang Chen, Kyle Xu, Yuchen Liu, Haifeng Wang, Zhenhao Shen, Jasper Lu, Shengze Huang, Yuanfei Wang, Chen Xie, and Ruihai Wu. Robowm-bench: A benchmark for evaluating world models in robotic manipulation. *arXiv preprint arXiv:2604.19092*, 2026.
- Junguang Jiang, Baixu Chen, Junwei Pan, Ximei Wang, Dapeng Liu, Jie Jiang, and Mingsheng Long. Forkmerge: Mitigating negative transfer in auxiliary-task learning. In *Advances in Neural Information Processing Systems*, 2023.

- Xingyu Lin, Harjatin Singh Baweja, George Kantor, and David Held. Adaptive auxiliary task weighting for reinforcement learning. In *Advances in Neural Information Processing Systems*, 2019.
- Clare Lyle, Mark Rowland, Georg Ostrovski, and Will Dabney. On the effect of auxiliary tasks on representation dynamics. *arXiv preprint arXiv:2102.13089*, 2021.
- Yu Shang, Zhuohang Li, Yiding Ma, Weikang Su, Xin Jin, Ziyu Wang, Xin Zhang, Yinzhou Tang, Chen Gao, Wei Wu, Xihui Liu, Dhruv Shah, Zhaoxiang Zhang, Zhibo Chen, Jun Zhu, Yonghong Tian, Tat-Seng Chua, Wenwu Zhu, and Yong Li. Worldarena: A unified benchmark for evaluating perception and functional utility of embodied world models. *arXiv preprint arXiv:2602.08971*, 2026.
- Özgür Şimşek and Andrew G. Barto. Skill characterization based on betweenness. In *Advances in Neural Information Processing Systems*, 2008.
- Richard S. Sutton, Doina Precup, and Satinder Singh. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112 (1–2):181–211, 1999.
- Shahram Najam Syed, Arthur Jakobsson, Haoran Hao, and Jeffrey Ichnowski. Intercepting the future: Latent-space predictive world model for dynamic vla manipulation. *arXiv preprint arXiv:2606.02486*, 2026.
- Chuan Wen, Jierui Lin, Trevor Darrell, Dinesh Jayaraman, and Yang Gao. Fighting copycat agents in behavioral cloning from observation histories. In *Advances in Neural Information Processing Systems*, 2020.
- Xiaoxu Xu, Hao Li, Jinhui Ye, Yilun Chen, Jia Zeng, Xinyi Chen, Linning Xu, Dahua Lin, Weixin Li, and Jiangmiao Pang. Futurevla: Joint visuomotor prediction for vision-language-action model. *arXiv preprint arXiv:2603.10712*, 2026.
- Yi Yang, Zhihong Liu, Siqi Kou, Yiyang Chen, Yanzhe Hu, Jianbo Zhou, Boyuan Zhao, Zhijie Wei, Xiao Xia, Xueqi Li, Pengfei Liu, and Zhijie Deng. World-language-action model for unified world modeling, language reasoning, and action synthesis. *arXiv preprint arXiv:2606.05979*, 2026.
- Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning. In *Advances in Neural Information Processing Systems*, 2020.
- Yang Yu, Shiyuan Zhang, Yifei Sheng, Haoxiang Ren, and Haoxin Lin. How should world models be evaluated for embodied decision-making? a decision-making-centric position. *arXiv preprint arXiv:2606.15032*, 2026.
- Weili Zeng, Yitong Xing, Fulong Liu, Chengqun Yang, Antao Xiang, Feng Tian, Jingnan Gao, Jisong Cai, Xin Wang, Xiaomin Wu, Yao Mu, and Yichao Yan. Enfold: Folding world-generator computation into predictive representations for efficient embodied control. *arXiv preprint arXiv:2607.26657*, 2026.
- Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunnan Wang, Xinqiang Yu, Jiazhaoh Zhang, Runpei Dong, Jiawei He, Fan Lu, He Wang, Zhizheng Zhang, Li Yi, Wenjun Zeng, and Xin Jin. Dreamvla: A vision-language-action model dreamed with comprehensive world knowledge. *arXiv preprint arXiv:2507.04447*, 2025.
- Yijie Zhu, Jie He, Rui Shao, Kaishen Yuan, Tao Tan, Xiaochen Yuan, and Zitong Yu. ΔVLA: Prior-guided vision-language-action models via world knowledge variation. *arXiv preprint arXiv:2603.08361*, 2026.

A APPENDIX

A.1 EFFECTIVE CONFIGURATIONS

The completed A0–A3 pilot shares `pi05_base`, joint-delta actions, quantile normalization, 20k updates, batch size 64, and a 50-action horizon. A0 sets `lmwm_hint_dim=0`. A1 and A2 use one prefix token with dimensions 768 and 1152. A3 uses dimension 1152, `lmwm_live_hint=True`, `lmwm_live_residual=True`, and milestone-loss weight 0.05. The confirmatory configuration instead uses absolute actions, mean/std normalization, batch 16, 50k updates, and the published optimizer schedule. The audited configuration source has SHA-256 `fe9377c2b93f2428...4dd7264`; the corrected A0 config is `pi05_robotwin_a0_public_recipe_bj`. It disables joint deltas and quantile normalization, uses the `robotwin2.0_absolute_meanstd` statistics, AdamW weight decay 0.01, batch 16, and 50k updates. Each future confirmatory arm must record its resolved config and dataset-manifest hash next to its checkpoint rather than relying on the mutable source file.

[EXP-TODO T0–T2] Insert immutable resolved configs, checkpoint IDs, training seeds, and dataset manifests for every confirmatory arm.

A.2 EVALUATION AND STATISTICS

Pilot simulator seed groups begin at 100k, 200k, 300k, and 400k. Candidate initializations are retained only when the expert planner and benchmark success check both pass. Each accepted cohort has 50 episodes per task and seed group. Because filtering can produce different accepted scenes, intervention tests use the intersection of scene identifiers. Existing A2 report aggregates are archived under `logs/eval_reports`; the evaluation script emits the accepted IDs used for each paired intersection. Confirmatory reports will include seed-level task cells, hierarchical bootstrap intervals, exact paired tests, and Holm-adjusted p -values. Independently filtered cohorts are never treated as matched observations.

[EXP-TODO E1] Publish accepted-scene intersections, per-seed task cells, hierarchical intervals, discordant counts, and Holm-adjusted tests.

A.3 RECURRENCE MINING

The mining script (SHA-256 `d05eefc2f9690163...62c`) applies the following deterministic procedure independently per task: (1) uniformly retain at most 200 successful episodes; (2) L2-normalize pooled DINOv3 features; (3) set the RBF bandwidth to the median cross-episode nearest-neighbor distance; (4) compute the average nearest-match recurrence score; (5) smooth with a temporal Gaussian of width 1.4; (6) detect valleys with prominence 0.03 and minimum distance $\max(2, \lfloor T/12 \rfloor)$; (7) choose the maximum-recurrence frame inside each resulting segment; and (8) pair every frame with the next segment’s representative, using the terminal frame in the final segment. Valley distance acts as temporal NMS, and selecting one ridge per segment performs within-episode deduplication. Cross-episode similarity enters the score rather than a separate global prototype-clustering stage.

An artifact audit exposed incomplete pilot coverage. The current `pairs.npz` (SHA-256 `5d68e21fbf0dd462...8ade8`) contains 335,624 pairs from five tasks and omits Stack-3 entirely:

Task	Episodes	Pairs	Unique targets	Future pairs
Hammer	200	22,820	260	99.12%
Stack-2	200	82,204	652	99.76%
Ranking RGB	200	91,836	829	99.78%
Ranking size	200	91,932	800	99.78%
Handover	200	46,832	339	99.57%
Stack-3	0	0	0	—

When a pair is absent, the data transform supplies the current image as a shape-compatible placeholder and sets the milestone-loss mask to false. The A3 predictor token remains

active through the action loss, but receives no milestone auxiliary supervision for that sample. Consequently the legacy Stack-3 A3 cell measures an added current-state predictor/capacity path, not a recurrence-supervised milestone. We therefore generated the frozen `robotwin_milestone_all6_confirmatory_v1` artifact. It contains 200 episodes for each of the six tasks (1,200 total), including 94,141 Stack-3 pairs; its `pairs.npz` SHA-256 begins `592f1e739105951e`, and its READY audit SHA-256 begins `4353c725f7e55f9c`. It passes nonzero coverage, monotone-target, selection-manifest, and source-hash checks and is the only recurrence artifact allowed for confirmatory A3 training. Threshold sensitivity and alternative selectors remain training experiments because their effect must be assessed through newly trained policies.

[EXP-TODO T4] Report mining-threshold sensitivity and alternative-selector comparisons from newly trained, protocol-matched policies.

A.4 LAWAM MECHANISM PROBE

All diagnostic variants inherit `lawam_pretrain`, which contains future prediction and distillation, and retain current DINO conditioning. The former “no-WM” row is therefore *LaWAM-init/Future-off*, not a clean VLA baseline. In the six-task matrix, Future-off, Local future, Dual absolute, Dual residual, Absolute+stop-grad, and Residual+stop-grad obtain 91.1, 86.6, 89.2, 87.9, 89.4, and 89.2%, respectively.

At fixed checkpoints, paired three-task interventions produce the following deltas relative to correct content:

Checkpoint	Correct	Zero Δ	Shuffle Δ	Cross-task Δ
Absolute	96.47	-0.93	+0.93	+0.19
Residual	91.64	+1.27	+2.00	+2.18
Residual + stop-grad	98.33	-1.49	-0.93	-1.12

The previously reported exact tests are unadjusted; multiplicity-aware results must replace them before submission. These data motivate $\pi_{0.5}$ content controls but do not answer Q2 for MINT-VLA.

A.5 EXPLORATORY TASK GROUPS AND LIBERO BOUNDARY

In the legacy Stack-2/3 subset, A0, A2-Abs, A2-Res, and A3 average 29.75, 49.75, 40.75, and 54.75%. The public no-hint checkpoint reaches 81.75%. This post-hoc two-task pattern is not a validated applicability law. The predeclared extension covers block and bowl stacking and includes reactive and geometric controls.

The auxiliary $\pi_{0.5}$ LIBERO diagnostic uses checkpoint step 29,999, OSMesa rendering, three rollout seeds, and the same one-prefix-token interface:

Suite	A0	A1 DINO	A2 So400m
LIBERO-10	94.7	94.3	92.7
Goal	96.7	97.3	97.3
Object	98.7	100.0	100.0
Spatial	99.7	99.3	98.7
Four-suite macro	97.42	97.75	97.18

The aggregate deltas are +0.33 and -0.24 points. Thirty of forty A0 tasks already exceed 97%, so this table is a saturation/boundary diagnostic rather than primary support for MINT-VLA. The task-type split observed inside LIBERO-10 (ordered closures positive, unordered collection negative) is exploratory and is tested prospectively in Q4 rather than promoted from these small cells.

A.6 EFFICIENCY AND REPRODUCIBILITY

Parameter trees constructed from the resolved Flax NNX models give the exact counts below. A3’s two 2048×2048 linear layers add 8.39M MACs (approximately 16.78 MFLOPs under the two-FLOP convention) per predictor call, excluding the small attention cost of one additional prefix token.

Arm	Parameters	Added vs. A0	Relative
A0	3,353,433,872	–	–
A1	3,355,008,784	1,574,912	0.047%
A2	3,355,795,216	2,361,344	0.070%
A3	3,361,826,576	8,392,704	0.250%

Production legacy A0, A1, and A3 runs each stabilized near 2.6 seconds per optimizer step with eight data workers; this coarse observation does not identify a model-specific throughput difference. A controlled one-GPU data path check measured 2.279 s/update for batch 16 with two accumulation steps and 2.142 s/update for batch 32 without accumulation. Peak memory and end-to-end action-server latency have not yet been measured under a paired protocol and are therefore not reported as advantages.

[EXP-TODO E2] Measure A0/A2/A3 peak train and inference memory, model latency, and end-to-end action-server throughput under one paired hardware protocol.

The implementation environment pins Python 3.11, JAX 0.5.3 with CUDA 12, Flax 0.10.2, and PyTorch 2.7.1. The audited workspace revision is 8d28087d2f69d063...2d01270e; because the experiment tree contains uncommitted research changes, file hashes above are the authoritative identifiers. Pilot rollout seed groups start at 100k, 200k, 300k, and 400k; independent training seeds and paired scene manifests will be attached to the confirmatory checkpoints. The AI-use disclosure appears in the main paper.

[EXP-TODO T0–T6,E1] Replace provisional identifiers with released checkpoint, scene-manifest, and per-episode artifact hashes.